

S.A.G.E. - Sistema di Analisi e Gestione Economica

Corso: Basi di Dati
Anno accademico 2025/2026

Componenti del gruppo

Antonio Scharmuller
Riccardo Dallacasa

Matricole

Matricola 000000
Matricola 000000

Email universitarie

antonio.scharmuller@studio.unibo.it
riccardo.dallacasa@studio.unibo.it

Docente: Nome Cognome

24 giugno 2026

Indice

1	Introduzione	1
1.1	Obiettivo del progetto	1
1.2	Descrizione generale del sistema	1
1.3	Ambito del sistema	1
1.4	Utenti e ruoli previsti	1
1.5	Tecnologie utilizzate	1
1.6	Scelte progettuali preliminari	2
2	Analisi dei requisiti	2
2.1	Requisiti in linguaggio naturale	2
2.2	Analisi delle ambiguità e decisioni progettuali	2
2.3	Specifiche ristrutturate	2
2.4	Glossario dei termini essenziali	3
2.5	Classi di utenza e viste del sistema	3
2.6	Elenco delle operazioni principali	3
2.7	Vincoli espliciti e impliciti	3
3	Progettazione concettuale	3
3.1	Criteri generali di modellazione	3
3.2	Entità principali	4
3.3	Progetto delle viste	4
3.3.1	Vista studente	4
3.3.2	Vista amministratore	5
3.3.3	Integrazione delle viste	6
3.4	Raffinamenti dello schema	6
3.4.1	Gerarchia delle transazioni	6
3.4.2	Elementi di sistema e personali	7
3.4.3	Reificazione del budget	7
3.4.4	Documenti digitali	7
3.5	Schema concettuale integrato dopo i raffinamenti	7
3.6	Associazioni principali e cardinalità	8
3.7	Vincoli non rappresentabili direttamente nello schema E/R	8
3.8	Dizionario di entità e associazioni	9
4	Specifiche funzionali	9
4.1	Elenco delle funzionalità richieste	9
4.2	Operazioni di inserimento	9
4.3	Operazioni di modifica ed eliminazione	9
4.4	Operazioni di interrogazione	9
4.5	Operazioni statistiche e amministrative	10
5	Progettazione logica	10
5.1	Stima del volume dei dati	10
5.2	Descrizione delle operazioni principali, frequenza e tabelle degli accessi	10
5.3	Analisi delle ridondanze	12
5.4	Raffinamento dello schema	13
5.4.1	Eliminazione delle gerarchie	13
5.4.2	Eliminazione degli attributi composti	13
5.4.3	Gestione degli attributi multivalore	13
5.4.4	Scelta delle chiavi primarie	13

5.4.5	Traduzione delle associazioni	13
5.5	Schema relazionale finale	13
5.6	Vincoli di integrità	14
5.7	Controllo della normalizzazione	14
6	Implementazione SQL	15
6.1	Script di creazione del database	15
6.2	Creazione delle tabelle	15
6.3	Definizione di chiavi primarie e chiavi esterne	16
6.4	Vincoli NOT NULL, UNIQUE, CHECK e indici	17
6.5	Trigger e viste	18
6.6	Popolamento iniziale del database	19
6.7	Migrazione di un database esistente	20
6.8	Query SQL delle operazioni principali	21
6.8.1	Problemi e criticità	21
6.8.2	Inserimento atomico di una spesa	21
6.8.3	Upsert del budget	22
6.8.4	Documenti dell'utente	22
6.8.5	Analisi percentuale	23
6.8.6	Query di supporto	23
6.8.7	Motivazioni	23
7	Progettazione dell'applicazione	24
7.1	Architettura generale dell'applicazione	24
7.2	Tecnologie usate	24
7.3	Struttura dei package Java	24
7.4	Collegamento al database tramite JDBC	25
7.5	Organizzazione delle classi principali	25
7.6	Gestione degli errori e validazione input	26
7.7	Interfaccia grafica Java Swing	26
7.8	Schermate principali	27
7.9	Flussi d'uso principali	27
8	Test e validazione	28
8.1	Dati di test utilizzati	28
8.2	Test delle query SQL	28
8.3	Test delle funzionalità applicative	28
8.4	Casi limite gestiti	29
8.5	Errori noti o limitazioni	29
9	Conclusioni	29
9.1	Risultato ottenuto	29
9.2	Scelte progettuali principali	29
9.3	Possibili sviluppi futuri	29

1 Introduzione

1.1 Obiettivo del progetto

S.A.G.E. (Sistema di Analisi e Gestione Economica) è un'applicazione desktop per la gestione delle finanze personali di uno studente universitario. Il sistema supporta la registrazione di entrate e spese, la classificazione dei movimenti, la definizione di budget e la consultazione di riepiloghi utili per monitorare l'andamento economico nel tempo.

L'obiettivo del progetto non è soltanto realizzare un'applicazione funzionante, ma documentare l'intero ciclo di progettazione di una base di dati: analisi dei requisiti, modellazione concettuale, traduzione logica, implementazione SQL e collegamento a un programma Java tramite JDBC.

1.2 Descrizione generale del sistema

S.A.G.E. è progettato come sistema multiutente. Ogni studente accede con le proprie credenziali e visualizza soltanto i dati collegati alla propria email. Il portafoglio personale comprende transazioni, categorie, fonti, tag, budget, documenti digitali e modelli di spese ricorrenti.

L'applicazione è realizzata in Java Swing e comunica con un database MySQL tramite JDBC. La logica applicativa è organizzata in livelli: le classi del package `model` rappresentano il dominio, i DAO incapsulano le query SQL, i controller coordinano i casi d'uso e le classi del package `view` compongono l'interfaccia grafica.

1.3 Ambito del sistema

L'ambito del progetto comprende:

- gestione individuale del portafoglio economico personale;
- registrazione di spese ed entrate;
- classificazione delle spese tramite categorie e tag;
- classificazione delle entrate tramite fonti;
- definizione di budget mensili e budget specifici per categoria;
- gestione di spese ricorrenti come modelli astratti;
- associazione di documenti digitali alle spese;
- visualizzazione di riepiloghi e statistiche.

Non rientrano nell'ambito del sistema la condivisione di conti tra utenti, la sincronizzazione cloud dei file allegati e l'integrazione con servizi bancari esterni.

1.4 Utenti e ruoli previsti

Il sistema prevede due ruoli:

- **Utente:** registra transazioni, consulta il proprio portafoglio, gestisce categorie personali, definisce budget e associa documenti alle spese.
- **Amministratore:** gestisce elementi di sistema, come categorie, fonti e tag globali. L'amministratore può accedere solo a statistiche aggregate e non ai dettagli finanziari personali degli utenti.

1.5 Tecnologie utilizzate

- **DBMS:** MySQL.
- **Ambiente locale:** XAMPP, utilizzato per avviare il server MySQL locale e per amministrare il database tramite phpMyAdmin durante sviluppo e test.
- **Linguaggio applicativo:** Java.
- **Interfaccia:** Java Swing.
- **Connessione al database:** JDBC con MySQL Connector/J.

- **Architettura:** applicazione desktop 2-tier con accesso diretto al DBMS.

1.6 Scelte progettuali preliminari

Le scelte iniziali più rilevanti sono:

- isolamento dei dati tramite email dell'utente autenticato;
- distinzione tra elementi globali di sistema ed elementi personali;
- gestione opzionale del budget, che non blocca l'inserimento di nuove spese;
- memorizzazione dei documenti digitali tramite percorso file, evitando di salvare il contenuto binario direttamente nel database.

La scelta sui documenti riduce il peso del DBMS e semplifica le operazioni JDBC. Presenta però un limite noto: se l'applicazione viene usata da un altro dispositivo collegato allo stesso database, un percorso locale potrebbe non essere più valido. Questa criticità viene considerata tra i possibili sviluppi futuri.

2 Analisi dei requisiti

2.1 Requisiti in linguaggio naturale

Il sistema deve permettere a uno studente di registrare e monitorare i propri movimenti economici. Ogni transazione deve avere data, importo e descrizione. Una spesa deve essere associata a una categoria e può avere tag e documenti digitali; un'entrata deve essere associata a una fonte. L'utente deve poter definire budget mensili riutilizzabili, globali o opzionalmente legati a una categoria. Il sistema deve inoltre supportare spese ricorrenti come modelli da cui generare movimenti effettivi.

2.2 Analisi delle ambiguità e decisioni progettuali

Concetto	Ambiguità	Decisione progettuale
Periodo temporale	Come collegare transazioni e budget mensili?	Il periodo è derivato dalla data della transazione; i budget sono unici per ambito e vengono ricalcolati sul periodo corrente.
Transazione	Separare fisicamente spese ed entrate?	Si usa una relazione unica TRANSIZIONE con attributo TipoTransazione .
Classificazione	Come distinguere catalogo comune e personalizzazioni?	CATEGORIA , TAG e FONTE usano is_system ed eventuale Email_Proprietario .
Spesa ricorrente	È una transazione già avvenuta?	No: è un modello generatore che può produrre transazioni future.
Documenti	Salvare file o riferimento?	Il DB salva percorso, tipo e data acquisizione. Il file resta nel file system, non in un BLOB.
Privacy	L'admin vede i movimenti personali?	No: l'accesso amministrativo è limitato a dati aggregati.

2.3 Specifiche ristrutturate

- R1: autenticare utenti e amministratori.
- R2: registrare spese ed entrate con data, importo e descrizione.

- R3: associare ogni spesa a una categoria e ogni entrata a una fonte.
- R4: associare tag e documenti digitali alle spese.
- R5: gestire budget mensili e budget per categoria.
- R6: gestire spese ricorrenti come modelli astratti.
- R7: filtrare ogni dato operativo tramite l'email dell'utente autenticato.
- R8: consentire all'amministratore solo operazioni su cataloghi globali e statistiche aggregate.

2.4 Glossario dei termini essenziali

- **Utente:** studente identificato da email, nome, cognome, password e ruolo.
- **Transazione:** movimento economico generico, distinto in spesa o entrata.
- **Categoria:** classificazione principale di una spesa.
- **Fonte:** origine di un'entrata.
- **Tag:** etichetta trasversale che consente classificazioni aggiuntive.
- **Budget:** limite di spesa mensile collegato a un utente e, opzionalmente, a una categoria; per lo stesso ambito esiste una sola configurazione.
- **Documento:** riferimento digitale a uno scontrino o a una fattura associata a una spesa.
- **Spesa ricorrente:** modello da cui generare spese periodiche.

2.5 Classi di utenza e viste del sistema

- **Studente:** utilizza dashboard, transazioni, budget, documenti, categorie personali e report.
- **Amministratore:** gestisce cataloghi globali e consulta indicatori aggregati.

La vista studente è centrata sul portafoglio personale. Ogni schermata applicativa riceve l'utente corrente e usa la sua email come chiave di sessione.

2.6 Elenco delle operazioni principali

- Login e apertura della dashboard personale.
- Inserimento di una spesa con categoria, tag opzionali e documento opzionale.
- Inserimento di una entrata con fonte.
- Consultazione dei movimenti per periodo.
- Definizione o aggiornamento di un budget.
- Associazione e apertura di un documento digitale.
- Creazione di categorie, fonti e tag personali.
- Consultazione di riepiloghi e statistiche aggregate.

2.7 Vincoli espliciti e impliciti

- Ogni query sui dati personali deve filtrare per email dell'utente autenticato.
- Una spesa deve avere una categoria e non deve avere una fonte.
- Una entrata deve avere una fonte e non deve avere una categoria.
- I tag sono associabili solo alle spese.
- I documenti sono associabili solo alle spese.
- Il budget è opzionale e non impedisce l'inserimento di movimenti.
- Il contenuto dei documenti non è salvato nel database; viene salvato solo il riferimento al file.

3 Progettazione concettuale

3.1 Criteri generali di modellazione

La progettazione concettuale privilegia un modello chiaro, con entità separate per gli oggetti persistenti principali e associazioni esplicite per i legami multi-a-molti. Le scelte sono guidate

da tre criteri: isolamento dei dati per utente, semplicità di interrogazione e coerenza con l'applicazione Java.

3.2 Entità principali

Entità	Ruolo nel sistema
UTENTE	Identifica studenti e amministratori. L'email è chiave primaria e riferimento per i dati personali.
TRANSIZIONE	Rappresenta movimenti economici, sia spese sia entrate, tramite TipoTransazione .
CATEGORIA	Classifica le spese; può essere di sistema o personale.
FONTE	Classifica le entrate; può essere di sistema o personale.
TAG	Aggiunge classificazioni trasversali alle spese.
PERIODO	Storicizza mese e anno usati dai budget.
BUDGET	Definisce una soglia mensile riutilizzata mese per mese, globale o per categoria.
DOCUMENTO	Conserva percorso, tipo e data di acquisizione del file associato a una spesa.
SPESA_RICORRENTE	Modello astratto per spese periodiche future.

3.3 Progetto delle viste

Come nelle relazioni di riferimento, lo schema viene prima presentato attraverso le viste parziali delle classi di utenza e successivamente integrato in uno schema globale. Le viste non introducono necessariamente nuove relazioni persistenti: descrivono soprattutto i dati e le operazioni rilevanti per ciascun ruolo.

3.3.1 Vista studente

La vista studente comprende il portafoglio personale: transazioni, periodi, categorie, fonti, tag, budget, documenti e spese ricorrenti. L'associazione con **UTENTE** rappresenta il vincolo fondamentale di isolamento dei dati.

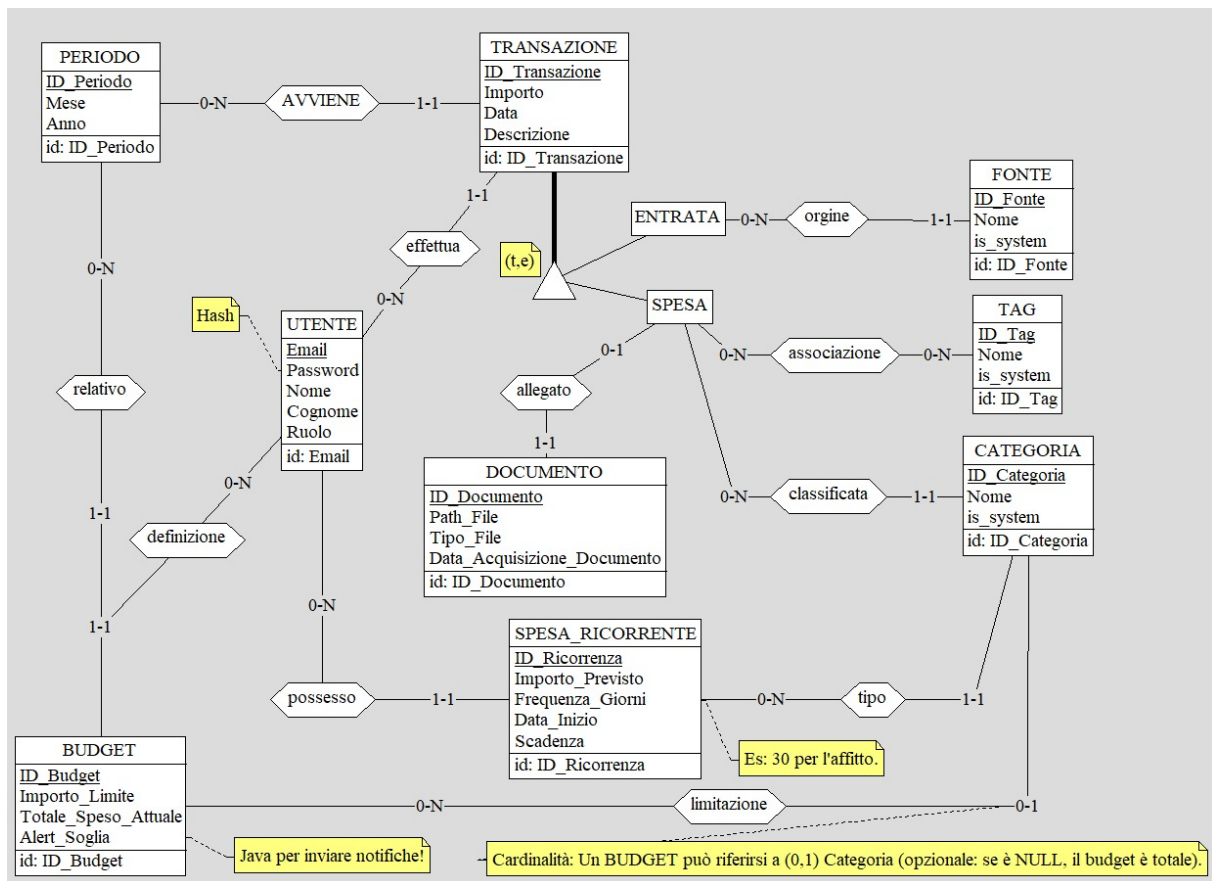


Figura 1: Vista concettuale dello studente.

3.3.2 Vista amministratore

La figura seguente rappresenta la parte persistente direttamente gestita dall'amministratore: categorie, tag e fonti di sistema. Lo schema è sufficiente come *vista funzionale* dei cataloghi globali, ma richiede due precisazioni. Nel database l'amministratore non è una entità autonoma: è una istanza di **UTENTE** con ruolo amministrativo. Inoltre le statistiche aggregate mostrate dalla dashboard non costituiscono una nuova entità, perché sono calcolate a partire da **TRANSAZIONE**, **PERIODO**, **CATEGORIA** e **BUDGET**. Non viene infine storicizzato quale amministratore abbia creato un elemento di sistema; per introdurre tale tracciamento sarebbe necessario aggiungere un riferimento all'utente amministratore nelle relative entità.

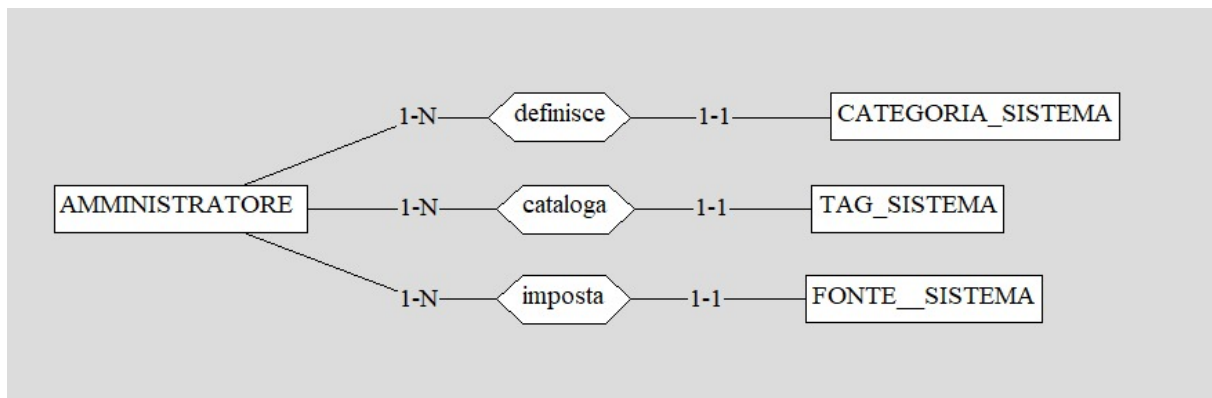


Figura 2: Vista funzionale dell'amministratore per la gestione dei cataloghi di sistema.

3.3.3 Integrazione delle viste

Le due viste condividono le classificazioni e i dati economici su cui vengono calcolate le statistiche. La loro integrazione produce lo schema ibrido riportato in Figura 3.

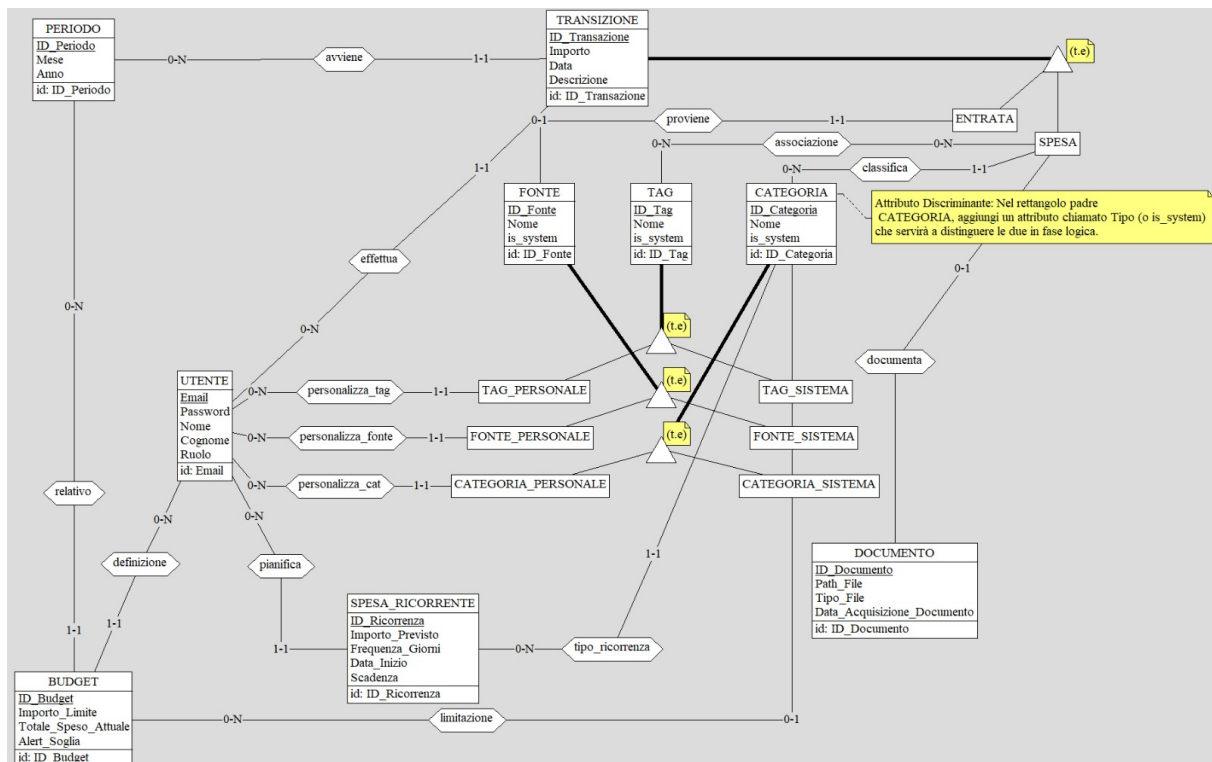


Figura 3: Integrazione delle viste studente e amministratore.

3.4 Raffinamenti dello schema

3.4.1 Gerarchia delle transazioni

Nel dominio esistono due sottotipi di transazione: spesa ed entrata. La specializzazione è totale ed esclusiva: ogni transazione è necessariamente una spesa o una entrata, e non può essere entrambe. Nel modello logico questa gerarchia è collasata in TRANSIZIONE, con l'attributo

TipoTransazione. I vincoli CHECK garantiscono la coerenza: una spesa richiede `ID_Categoria`, una entrata richiede `ID_Fonte`.

3.4.2 Elementi di sistema e personali

CATEGORIA, TAG e FONTE sono modellati con una soluzione ibrida. Gli elementi globali hanno `is_system = TRUE` e nessun proprietario; gli elementi personali hanno `is_system = FALSE` e sono collegati a un utente. Le query applicative mostrano a uno studente sia gli elementi globali sia quelli personali.

3.4.3 Reificazione del budget

Il budget è modellato come entità autonoma collegata all'utente e a un ambito: globale oppure riferito a una categoria. Il sistema mantiene una sola configurazione per ciascun ambito e aggiorna il periodo di riferimento al mese corrente; in questo modo non vengono create copie gennaio, febbraio, marzo dello stesso budget. L'attributo `Totale_Speso_Attuale` è ridondante, ma serve a rendere efficiente la dashboard.

3.4.4 Documenti digitali

Il documento digitale è collegato a una spesa e contiene `Path_File`, `Tipo_File` e `Data_Acquisizione_Documento`. Il contenuto binario del file non è salvato nel DBMS.

Questa scelta riduce il peso del database e semplifica backup, query e codice JDBC. Il limite principale è la portabilità: spostandosi su un altro dispositivo, il percorso locale potrebbe non esistere. Una futura versione potrebbe usare path relativi sincronizzati oppure un attributo `LOB`, qualora la portabilità completa del documento attraverso il solo database diventasse un requisito prioritario.

3.5 Schema concettuale integrato dopo i raffinamenti

Lo schema seguente recepisce il collasso delle specializzazioni di categoria, tag e fonte mediante gli attributi `is_system` ed `Email_creatore`, oltre al discriminante della transazione. Il termine “normalizzazione” usato nel nome originario dell'immagine indica qui il raffinamento concettuale; la verifica delle forme normali del modello relazionale viene svolta successivamente, nella progettazione logica.

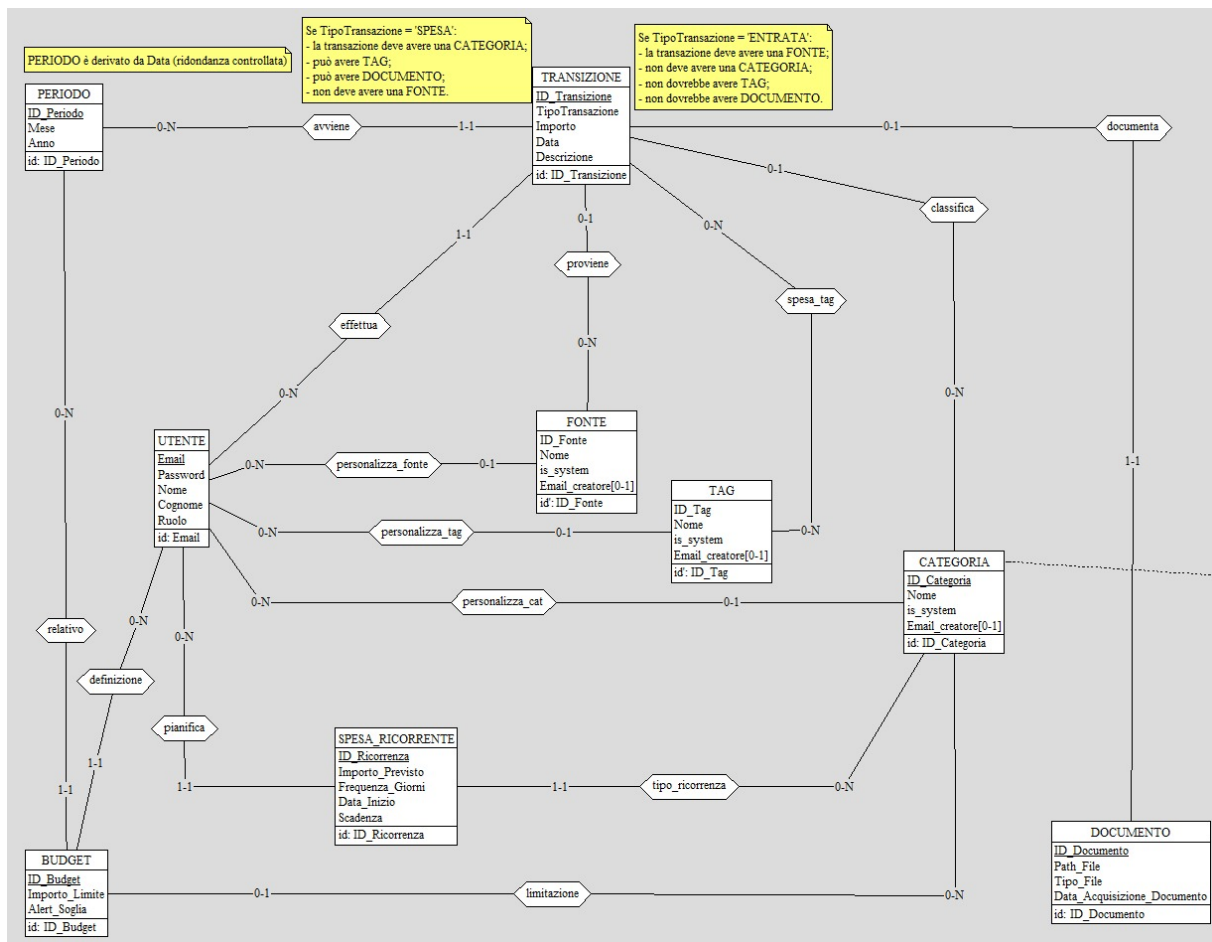


Figura 4: Schema concettuale integrato dopo i raffinamenti.

3.6 Associazioni principali e cardinalità

Associazione	Cardinalità	Motivazione
Utente–Transazione	1:N	Un utente può registrare molte transazioni; ogni transazione appartiene a un solo utente.
Transazione–Categoria	N:1 opzionale	Solo le spese hanno una categoria.
Transazione–Fonte	N:1 opzionale	Solo le entrate hanno una fonte.
Spesa–Tag	N:M	Una spesa può avere più tag e un tag può essere riutilizzato.
Spesa–Documento	1:N logico	Una spesa può avere documenti; nella UI corrente si limita a un documento per semplicità.
Utente–Budget	1:N	Ogni utente può definire più budget, ma al massimo uno per ambito.
Periodo–Budget	1:N	Il periodo indica il mese di riferimento corrente usato per calcolare lo stato del budget.

3.7 Vincoli non rappresentabili direttamente nello schema E/R

Alcune regole richiedono vincoli SQL o controlli applicativi:

- una categoria personale è visibile solo al proprietario;
- una spesa può usare solo categorie di sistema o categorie personali dell'utente;
- un documento può essere collegato solo a una spesa, non a una entrata;
- l'amministratore non deve visualizzare dettagli finanziari personali;
- l'inserimento di spesa, tag, documento e aggiornamento budget deve essere atomico.

3.8 Dizionario di entità e associazioni

Il dizionario riassume gli attributi principali:

- UTENTE: Email, Password, Nome, Cognome, Ruolo.
- TRANSIZIONE: ID, TipoTransazione, Importo, Data, Descrizione, Email, Categoria/Fonte.
- BUDGET: ID, Importo_Limite, Alert_Soglia, Totale_Speso_Attuale, Periodo corrente, Categoria opzionale, Email; il vincolo su Email e categoria impedisce duplicati dello stesso budget.
- DOCUMENTO: ID, ID_Transizione, Path_File, Tipo_File, Data_Acquisizione_Documento.
- SPESA_TAG: associazione multi-a-molti tra spesa e tag.

4 Specifiche funzionali

4.1 Elenco delle funzionalità richieste

Le funzionalità principali sono organizzate in operazioni utente e operazioni amministrative. Le prime lavorano sempre sul portafoglio dello studente autenticato; le seconde riguardano cataloghi e statistiche aggregate.

4.2 Operazioni di inserimento

Operazione	Input	Tabelle coinvolte
Login	Email, password	UTENTE
Nuova spesa	Importo, data, descrizione, categoria, tag, documento opzionale	PERIODO, TRANSIZIONE, SPESA_TAG, DOCUMENTO, BUDGET
Nuova entrata	Importo, data, descrizione, fonte	PERIODO, TRANSIZIONE
Nuovo budget	Importo limite, categoria opzionale	BUDGET, PERIODO, TRANSIZIONE
Nuovo documento	Spesa, percorso file, tipo file	DOCUMENTO, TRANSIZIONE

4.3 Operazioni di modifica ed eliminazione

Le modifiche riguardano budget, categorie, fonti e tag, sia personali sia di sistema quando l'operazione è svolta dall'amministratore. Le eliminazioni devono rispettare i vincoli referenziali: ad esempio una categoria eliminata non deve lasciare riferimenti incoerenti nelle spese già registrate. La cancellazione di transazioni o documenti sfrutta le regole di integrità definite nello schema SQL.

4.4 Operazioni di interrogazione

- elenco delle transazioni dell'utente per periodo;
- elenco di categorie, fonti e tag disponibili;
- elenco dei budget e relativo stato di superamento;

- elenco dei documenti associati alle spese dell'utente;
- elenco delle spese ancora prive di documento.

4.5 Operazioni statistiche e amministrative

Le statistiche includono totali mensili di spese ed entrate, rapporto percentuale tra spese ed entrate, budget superati e aggregazioni per categoria. L'amministratore deve utilizzare viste aggregate, evitando l'accesso a descrizioni, documenti o movimenti granulari degli utenti.

5 Progettazione logica

5.1 Stima del volume dei dati

La tabella seguente riporta il numero medio di istanze previsto per ogni entità e associazione dello schema concettuale. Le stime descrivono un utilizzo pluriennale del sistema da parte di circa 300 utenti e costituiscono la base per il calcolo dei costi delle operazioni.

Tabella 5: Stima del volume dei dati

NOME	TIPO	VOLUME DI DATI
UTENTE	E	300
PERIODO	E	40
TRANSIZIONE	E	120.000
SPESA	E	100.000
ENTRATA	E	20.000
CATEGORIA	E	2.020
CATEGORIA_PERSONALE	E	2.000
CATEGORIA_SISTEMA	E	20
TAG	E	2.510
TAG_PERSONALE	E	2.500
TAG_SISTEMA	E	10
FONTE	E	1.010
FONTE_PERSONALE	E	1.000
FONTE_SISTEMA	E	10
DOCUMENTO	E	15.000
BUDGET	E	10.000
SPESA_RICORRENTE	E	1.000
EFFETTUA	R	120.000
AVVIENE	R	120.000
PROVIENE	R	20.000
CLASSIFICA	R	100.000
ASSOCIAZIONE	R	100.000
DOCUMENTA	R	15.000
PERSONALIZZA_CAT	R	2.000
PERSONALIZZA_TAG	R	2.500
PERSONALIZZA_FONTE	R	1.000
DEFINIZIONE	R	10.000
RELATIVO	R	10.000
LIMITAZIONE	R	8.000
PIANIFICA	R	1.000
TIPO_RICORRENZA	R	1.000

Le stime risultano internamente coerenti: le 120.000 transazioni sono ripartite in 100.000 spese e 20.000 entrate; categorie, tag e fonti coincidono con la somma dei rispettivi elementi personali e di sistema. Il volume di ASSOCIAZIONE assume in media un tag per spesa; DOCUMENTA indica che il 15% delle spese possiede un documento; LIMITAZIONE indica che 8.000 budget su 10.000 sono specifici per categoria, mentre i restanti sono budget generali.

5.2 Descrizione delle operazioni principali, frequenza e tabelle degli accessi

Le operazioni derivano dai casi d'uso descritti nell'analisi dei requisiti. Tutte le frequenze sono espresse su base mensile e sono già incorporate nella colonna *Totale degli accessi al mese*. Le

letture sono indicate con L e le scritture con S. Nel calcolo del costo pesato, una scrittura vale due letture; la tabella riporta invece il numero grezzo di accessi, in modo da rendere immediatamente verificabile il prodotto tra frequenza e accessi per singola esecuzione.

Tabella 6: Operazioni principali e tabella degli accessi

Operazione	Concetto	Entità/ Assoc.	Lettura/ Scrittura	Numero di accessi	Totale degli accessi (al mese)
1 – Registrazione utente	UTENTE	E	S	1	30
2.1 – Inserimento di una nuova spesa	UTENTE	E	L	1	2.500
	PERIODO	E	L	1	2.500
	TRANSIZIONE	E	S	1	2.500
	SPESA	E	S	1	2.500
	EFFETTUA	R	S	1	2.500
	AVVIENE	R	S	1	2.500
	CATEGORIA	E	L	1	2.500
	CLASSIFICA	R	S	1	2.500
	TAG	E	L	1	2.500
2.2 – Inserimento di una nuova entrata	ASSOCIAZIONE	R	S	1	2.500
	UTENTE	E	L	1	600
	PERIODO	E	L	1	600
	TRANSIZIONE	E	S	1	600
	ENTRATA	E	S	1	600
	EFFETTUA	R	S	1	600
	AVVIENE	R	S	1	600
	FONTE	E	L	1	600
	PROVIENE	R	S	1	600
3 – Visualizzazione delle transazioni di un periodo	UTENTE	E	L	1	3.000
	PERIODO	E	L	1	3.000
	EFFETTUA	R	L	10	30.000
	AVVIENE	R	L	10	30.000
	TRANSIZIONE	E	L	10	30.000
	SPESA	E	L	8	24.000
	ENTRATA	E	L	2	6.000
	CLASSIFICA	R	L	8	24.000
	CATEGORIA	E	L	8	24.000
	PROVIENE	R	L	2	6.000
	FONTE	E	L	2	6.000
	DOCUMENTA	R	L	1	3.000
	DOCUMENTO	E	L	1	3.000
	ASSOCIAZIONE	R	L	8	24.000
	TAG	E	L	8	24.000
4 – Analisi delle spese per categoria e tag	UTENTE	E	L	1	1.500
	PERIODO	E	L	2	3.000
	EFFETTUA	R	L	20	30.000
	AVVIENE	R	L	20	30.000
	TRANSIZIONE	E	L	20	30.000
	SPESA	E	L	20	30.000
	CLASSIFICA	R	L	20	30.000
	CATEGORIA	E	L	10	15.000
	ASSOCIAZIONE	R	L	20	30.000
5 – Definizione o modifica di un budget	TAG	E	L	10	15.000
	UTENTE	E	L	1	1.000
	PERIODO	E	L	1	1.000
	BUDGET	E	S	1	1.000
	DEFINIZIONE	R	S	1	1.000
	RELATIVO	R	S	1	1.000
6 – Monitoraggio del superamento del budget	CATEGORIA	E	L	1	1.000
	LIMITAZIONE	R	S	1	1.000
	UTENTE	E	L	1	6.000
	PERIODO	E	L	1	6.000
	DEFINIZIONE	R	L	5	30.000
	RELATIVO	R	L	5	30.000
7 – Definizione di una spesa ricorrente	BUDGET	E	L	5	30.000
	LIMITAZIONE	R	L	4	24.000
	CATEGORIA	E	L	4	24.000
	UTENTE	E	L	1	100
	SPESA_RICORRENTE	E	S	1	100
	PIANIFICA	R	S	1	100
8 – Generazione mensile delle spese ricorrenti	CATEGORIA	E	L	1	100
	TIPO_RICORRENZA	R	S	1	100
	SPESA_RICORRENTE	E	L	1	1.000
	PIANIFICA	R	L	1	1.000
	UTENTE	E	L	1	1.000
	TIPO_RICORRENZA	R	L	1	1.000
	CATEGORIA	E	L	1	1.000
	PERIODO	E	L	1	1.000
	TRANSIZIONE	E	S	1	1.000
	SPESA	E	S	1	1.000
	EFFETTUA	R	S	1	1.000
	AVVIENE	R	S	1	1.000
	CLASSIFICA	R	S	1	1.000
	BUDGET	E	S	2	2.000

Operazione	Concetto	Entità/ Assoc.	Lettura/ Scrittura	Numero di accessi	Totale degli accessi (al mese)
9 – Associazione di un documento a una spesa	SPESA	E	L	1	300
	DOCUMENTO	E	S	1	300
	DOCUMENTA	R	S	1	300
10.1 – Creazione o modifica di una categoria personale	UTENTE	E	L	1	300
	CATEGORIA	E	S	1	300
	CATEGORIA_PERSONALE	E	S	1	300
	PERSONALIZZA_CAT	R	S	1	300
10.2 – Creazione o modifica di un tag personale	UTENTE	E	L	1	400
	TAG	E	S	1	400
	TAG_PERSONALE	E	S	1	400
	PERSONALIZZA_TAG	R	S	1	400
10.3 – Creazione o modifica di una fonte personale	UTENTE	E	L	1	200
	FONTE	E	S	1	200
	FONTE_PERSONALE	E	S	1	200
	PERSONALIZZA_FONTE	R	S	1	200
11.1 – Gestione amministrativa delle categorie di sistema	UTENTE	E	L	1	20
	CATEGORIA	E	S	1	20
	CATEGORIA_SISTEMA	E	S	1	20
11.2 – Gestione amministrativa dei tag di sistema	UTENTE	E	L	1	20
	TAG	E	S	1	20
	TAG_SISTEMA	E	S	1	20
11.3 – Gestione amministrativa delle fonti di sistema	UTENTE	E	L	1	10
	FONTE	E	S	1	10
	FONTE_SISTEMA	E	S	1	10
12 – Visualizzazione di statistiche aggregate anonime	TRANSIZIONE	E	L	3.000	30.000
	SPESA	E	L	2.500	25.000
	ENTRATA	E	L	500	5.000
	AVVIENE	R	L	3.000	30.000
	CLASSIFICA	R	L	2.500	25.000
	CATEGORIA	E	L	2.000	20.000
	BUDGET	E	L	1.000	10.000
	LIMITAZIONE	R	L	800	8.000

La navigazione parte normalmente da UTENTE, identificato dalla sessione. Le transazioni vengono raggiunte tramite EFFETTUA e sono collocate temporalmente tramite AVVIENE. Da una spesa si prosegue verso categoria, tag e documento; da una entrata verso la fonte. I budget sono raggiunti tramite DEFINIZIONE e RELATIVO, mentre LIMITAZIONE è presente soltanto per i budget specifici di categoria. Le spese ricorrenti sono raggiunte tramite PIANIFICA e collegate alla categoria mediante TIPO_RICORRENZA.

Le ipotesi usate sono: dieci transazioni medie nella consultazione di un periodo, di cui otto spese e due entrate; un tag medio per spesa; un documento ogni dieci movimenti consultati; cinque budget mostrati nella dashboard, di cui quattro specifici per categoria. Per il monitoraggio non vengono rilette tutte le spese, perché si sfrutta la ridondanza Totale_Speso_Attuale. Nella generazione delle ricorrenze si considerano al massimo due budget aggiornabili: quello generale e quello della categoria.

5.3 Analisi delle ridondanze

L'attributo Totale_Speso_Attuale della relazione BUDGET è ridondante perché può essere ricavato sommando le spese presenti in TRANSIZIONE. La ridondanza viene mantenuta per ridurre il costo delle letture più frequenti della dashboard.

La coerenza viene garantita con due regole applicative:

- quando viene inserita una spesa, l'inserimento della transazione, l'associazione di tag/-documento e l'aggiornamento degli eventuali budget avvengono nella stessa transazione JDBC;
- quando un budget viene creato o aggiornato dopo spese già registrate, il valore iniziale viene calcolato con una query di aggregazione sulle transazioni esistenti.

In questo modo il budget rimane opzionale senza perdere consistenza: l'assenza di un budget non blocca la registrazione della spesa, mentre l'esistenza di un budget impone la sincronizzazione del suo totale ridondante.

5.4 Raffinamento dello schema

5.4.1 Eliminazione delle gerarchie

La gerarchia spesa/entrata viene eliminata collassando i sottotipi in TRANSIZIONE. La distinzione rimane espressa da `TipoTransazione`, mentre i vincoli CHECK controllano la presenza corretta di categoria o fonte.

5.4.2 Eliminazione degli attributi composti

Non sono presenti attributi composti significativi da scomporre ulteriormente. Nome e cognome dell'utente sono già separati; il periodo è rappresentato da mese e anno nella tabella PERIODO.

5.4.3 Gestione degli attributi multivalore

I tag associati a una spesa rappresentano un attributo multivalore. Vengono tradotti nella relazione SPESA_TAG, con chiave primaria composta da `ID_Transizione` e `ID_Tag`.

5.4.4 Scelta delle chiavi primarie

L'email è scelta come chiave primaria di UTENTE, perché identifica direttamente la sessione applicativa. Le altre entità principali utilizzano identificatori numerici auto-incrementali per semplificare riferimenti e join.

5.4.5 Traduzione delle associazioni

Le associazioni 1:N sono tradotte tramite chiavi esterne. Le associazioni N:M, come quella tra spese e tag, sono tradotte tramite tabella ponte. I vincoli applicativi più specifici sono rafforzati tramite trigger e controlli SQL.

5.5 Schema relazionale finale

Lo schema relazionale finale è riportato in Figura 5 e comprende dieci relazioni persistenti. La struttura è coerente con lo script ufficiale `doc/sql/01_schema_completo.sql`, che definisce chiavi primarie, chiavi esterne, vincoli CHECK e indici.

Attuale in BUDGET, documentata e controllata tramite logica applicativa.

6 Implementazione SQL

6.1 Script di creazione del database

Gli script SQL consolidati sono 01_schema_completo.sql, 02_popolamento_demo.sql, 03_migrazione_databa e 04_query_operazioni.sql.

Lo script doc/sql/01_schema_completo.sql crea da zero il database MySQL e seleziona lo schema prima di definire le relazioni. L'ordine di creazione segue le dipendenze referenziali: prima le entità indipendenti, poi quelle che importano chiavi esterne.

```
1 DROP DATABASE IF EXISTS
  Schema_finale_del_relazionale_SAGE_Vista_ibrida_Raffinata;
2 CREATE DATABASE
  Schema_finale_del_relazionale_SAGE_Vista_ibrida_Raffinata;
3 USE Schema_finale_del_relazionale_SAGE_Vista_ibrida_Raffinata;
4
5 CREATE TABLE UTENTE (
6   Email VARCHAR(255) NOT NULL,
7   Password VARCHAR(128) NOT NULL,
8   Nome VARCHAR(80) NOT NULL,
9   Cognome VARCHAR(80) NOT NULL,
10  Ruolo VARCHAR(10) NOT NULL DEFAULT 'UTENTE',
11  CONSTRAINT PK_UTENTE PRIMARY KEY (Email),
12  CONSTRAINT CK_UTENTE_RUOLO
13    CHECK (Ruolo IN ('UTENTE', 'ADMIN'))
14 );
15
16 CREATE TABLE PERIODO (
17   ID_Periodo INT NOT NULL AUTO_INCREMENT,
18   Mese TINYINT NOT NULL,
19   Anno SMALLINT NOT NULL,
20   CONSTRAINT PK_PERIODO PRIMARY KEY (ID_Periodo),
21   CONSTRAINT UQ_PERIODO_MESE_ANNO UNIQUE (Mese, Anno),
22   CONSTRAINT CK_PERIODO_MESE CHECK (Mese BETWEEN 1 AND 12),
23   CONSTRAINT CK_PERIODO_ANNO CHECK (Anno >= 2000)
24 );
```

Listing 1: Creazione dello schema e delle tabelle UTENTE e PERIODO

6.2 Creazione delle tabelle

Le tabelle di classificazione adottano la soluzione ibrida descritta nello schema concettuale. Il campo `is_system` distingue gli elementi globali da quelli personali, mentre `Email_Proprietario` è valorizzata soltanto per gli elementi creati dall'utente. `FONTE` e `TAG` sono costruite con la stessa struttura di `CATEGORIA`.

```
1 CREATE TABLE CATEGORIA (
2   ID_Categoria INT NOT NULL AUTO_INCREMENT,
3   Nome VARCHAR(80) NOT NULL,
4   Icona VARCHAR(255),
5   is_system BOOLEAN NOT NULL DEFAULT FALSE,
6   Email_Proprietario VARCHAR(255),
7   CONSTRAINT PK_CATEGORIA PRIMARY KEY (ID_Categoria),
8   CONSTRAINT FK_CATEGORIA_UTENTE
9     FOREIGN KEY (Email_Proprietario)
10    REFERENCES UTENTE (Email)
```

```

11         ON UPDATE CASCADE
12         ON DELETE CASCADE,
13     CONSTRAINT CK_CATEGORIA_OWNER CHECK (
14         (is_system = TRUE AND Email_Proprietario IS NULL)
15         OR
16         (is_system = FALSE AND Email_Proprietario IS NOT NULL)
17     )
18 );

```

Listing 2: Tabella CATEGORIA per elementi di sistema e personali

La relazione TRANSIZIONE realizza il collasso della gerarchia tra spesa ed entrata. Il vincolo CK_TRANSIZIONE_COLLASSO impone una categoria alle spese e una fonte alle entrate, escludendo le combinazioni incoerenti.

```

1 CREATE TABLE TRANSIZIONE (
2     ID_Transizione INT NOT NULL AUTO_INCREMENT,
3     TipoTransazione CHAR(1) NOT NULL,
4     Importo DECIMAL(10,2) NOT NULL,
5     Data DATE NOT NULL,
6     Descrizione VARCHAR(255) NOT NULL,
7     Email VARCHAR(255) NOT NULL,
8     ID_Categoria INT,
9     ID_Periodo INT NOT NULL,
10    ID_Fonte INT,
11    ID_Ricorrenza INT,
12    CONSTRAINT PK_TRANSIZIONE PRIMARY KEY (ID_Transizione),
13    CONSTRAINT FK_TRANSIZIONE_UTENTE FOREIGN KEY (Email)
14        REFERENCES UTENTE (Email) ON UPDATE CASCADE ON DELETE CASCADE,
15    CONSTRAINT FK_TRANSIZIONE_CATEGORIA FOREIGN KEY (ID_Categoria)
16        REFERENCES CATEGORIA (ID_Categoria) ON UPDATE CASCADE ON DELETE
17    SET NULL,
18    CONSTRAINT FK_TRANSIZIONE_PERIODO FOREIGN KEY (ID_Periodo)
19        REFERENCES PERIODO (ID_Periodo) ON UPDATE CASCADE ON DELETE
20    RESTRICT,
21    CONSTRAINT FK_TRANSIZIONE_FONTE FOREIGN KEY (ID_Fonte)
22        REFERENCES FONTE (ID_Fonte) ON UPDATE CASCADE ON DELETE SET NULL
23    ,
24    CONSTRAINT CK_TRANSIZIONE_TIPO CHECK (TipoTransazione IN ('S', 'E'))
25    ,
26    CONSTRAINT CK_TRANSIZIONE_IMPORTO CHECK (Importo > 0),
27    CONSTRAINT CK_TRANSIZIONE_RICORRENZA CHECK (
28        ID_Ricorrenza IS NULL OR TipoTransazione = 'S'
29    ),
30    CONSTRAINT CK_TRANSIZIONE_COLLASSO CHECK (
31        (TipoTransazione = 'S' AND ID_Categoria IS NOT NULL AND ID_Fonte

```

Listing 3: Tabella TRANSIZIONE e vincolo sul tipo di movimento

6.3 Definizione di chiavi primarie e chiavi esterne

Ogni tabella ha una chiave primaria esplicita. Le associazioni molti-a-molti diventano relazioni autonome, come SPESA_TAG; i documenti e le ricorrenze importano invece la chiave della

transazione o dell'utente interessato. La chiave esterna verso SPESA_RICORRENTE viene aggiunta dopo la creazione di entrambe le tabelle per rispettare l'ordine delle dipendenze.

```
1 CREATE TABLE SPESA_TAG (  
2     ID_Transizione INT NOT NULL,  
3     ID_Tag INT NOT NULL,  
4     CONSTRAINT PK_SPESA_TAG PRIMARY KEY (ID_Transizione, ID_Tag),  
5     CONSTRAINT FK_SPESA_TAG_TRANSIZIONE FOREIGN KEY (ID_Transizione)  
6         REFERENCES TRANSIZIONE (ID_Transizione)  
7         ON UPDATE CASCADE ON DELETE CASCADE,  
8     CONSTRAINT FK_SPESA_TAG_TAG FOREIGN KEY (ID_Tag)  
9         REFERENCES TAG (ID_Tag)  
10        ON UPDATE CASCADE ON DELETE CASCADE  
11 );  
12  
13 CREATE TABLE DOCUMENTO (  
14     ID_Documento INT NOT NULL AUTO_INCREMENT,  
15     ID_Transizione INT NOT NULL,  
16     Path_File VARCHAR(500) NOT NULL,  
17     Tipo_File VARCHAR(20) NOT NULL,  
18     Data_Acquisizione_Documento DATE NOT NULL,  
19     CONSTRAINT PK_DOCUMENTO PRIMARY KEY (ID_Documento),  
20     CONSTRAINT FK_DOCUMENTO_TRANSIZIONE FOREIGN KEY (ID_Transizione)  
21         REFERENCES TRANSIZIONE (ID_Transizione)  
22        ON UPDATE CASCADE ON DELETE CASCADE  
23 );  
24  
25 CREATE TABLE SPESA_RICORRENTE (  
26     ID_Ricorrenza INT NOT NULL AUTO_INCREMENT,  
27     Nome VARCHAR(100) NOT NULL,  
28     Importo_Previsto DECIMAL(10,2) NOT NULL,  
29     Frequenza_Giorni INT NOT NULL,  
30     Data_Inizio DATE NOT NULL,  
31     Data_Prossima_Scadenza DATE NOT NULL,  
32     Scadenza DATE,  
33     ID_Categoria INT NOT NULL,  
34     Email VARCHAR(255) NOT NULL,  
35     CONSTRAINT PK_SPESA_RICORRENTE PRIMARY KEY (ID_Ricorrenza),  
36     CONSTRAINT FK_RICORRENTE_CATEGORIA FOREIGN KEY (ID_Categoria)  
37         REFERENCES CATEGORIA (ID_Categoria) ON DELETE RESTRICT,  
38     CONSTRAINT FK_RICORRENTE_UTENTE FOREIGN KEY (Email)  
39         REFERENCES UTENTE (Email) ON DELETE CASCADE,  
40     CONSTRAINT CK_RICORRENTE_IMPORTO CHECK (Importo_Previsto > 0),  
41     CONSTRAINT CK_RICORRENTE_FREQUENZA CHECK (Frequenza_Giorni > 0)  
42 );  
43  
44 ALTER TABLE TRANSIZIONE  
45     ADD CONSTRAINT FK_TRANSIZIONE_RICORRENZA  
46     FOREIGN KEY (ID_Ricorrenza)  
47     REFERENCES SPESA_RICORRENTE (ID_Ricorrenza)  
48     ON UPDATE CASCADE  
49     ON DELETE SET NULL;
```

Listing 4: Relazioni SPESA_TAG, DOCUMENTO e SPESA_RICORRENTE

6.4 Vincoli NOT NULL, UNIQUE, CHECK e indici

I vincoli NOT NULL garantiscono gli attributi essenziali, i vincoli UNIQUE impediscono duplicati logici e i CHECK formalizzano le regole di dominio. Gli indici aggiunti corrispondono alle ricerche

più frequenti dell'applicazione: movimenti di un utente ordinati per data, transazioni di un periodo, documenti collegati a una sola spesa e rate generate da una ricorrenza.

```

1 CREATE INDEX IDX_TRANSIZIONE_UTENTE_DATA
2   ON TRANSIZIONE (Email, Data);
3 CREATE INDEX IDX_TRANSIZIONE_PERIODO
4   ON TRANSIZIONE (ID_Periodo);
5 CREATE INDEX IDX_TRANSIZIONE_CATEGORIA
6   ON TRANSIZIONE (ID_Categoria);
7 CREATE UNIQUE INDEX UQ_TRANSIZIONE_RICORRENZA_DATA
8   ON TRANSIZIONE (ID_Ricorrenza, Data);
9 CREATE INDEX IDX_BUDGET_UTENTE_PERIODO
10  ON BUDGET (Email, ID_Periodo);
11 CREATE UNIQUE INDEX UQ_DOCUMENTO_TRANSIZIONE
12  ON DOCUMENTO (ID_Transizione);
13 CREATE INDEX IDX_RICORRENTE_EMAIL_SCADENZA
14  ON SPESA_RICORRENTE (Email, Data_Prossima_Scadenza);

```

Listing 5: Indici principali dello schema

6.5 Trigger e viste

I trigger rafforzano i vincoli che devono valere anche al di fuori dell'applicazione Java. Il seguente controllo impedisce che una transazione utilizzi una categoria o una fonte non visibile all'utente e verifica la coerenza del collegamento con una ricorrenza.

```

1 DELIMITER $$
2 CREATE TRIGGER trg_transizione_insert_check
3 BEFORE INSERT ON TRANSIZIONE
4 FOR EACH ROW
5 BEGIN
6   IF NEW.TipoTransazione = 'S' AND NOT EXISTS (
7     SELECT 1 FROM CATEGORIA C
8     WHERE C.ID_Categoria = NEW.ID_Categoria
9           AND (C.is_system = TRUE OR C.Email_Proprietario = NEW.Email)
10  ) THEN
11     SIGNAL SQLSTATE '45000'
12     SET MESSAGE_TEXT = 'Categoria non valida per questo utente';
13  END IF;
14
15  IF NEW.TipoTransazione = 'E' AND NOT EXISTS (
16    SELECT 1 FROM FONTE F
17    WHERE F.ID_Fonte = NEW.ID_Fonte
18          AND (F.is_system = TRUE OR F.Email_Proprietario = NEW.Email)
19  ) THEN
20     SIGNAL SQLSTATE '45000'
21     SET MESSAGE_TEXT = 'Fonte non valida per questo utente';
22  END IF;
23
24  IF NEW.ID_Ricorrenza IS NOT NULL AND NOT EXISTS (
25    SELECT 1 FROM SPESA_RICORRENTE SR
26    WHERE SR.ID_Ricorrenza = NEW.ID_Ricorrenza
27          AND SR.Email = NEW.Email
28          AND SR.ID_Categoria = NEW.ID_Categoria
29  ) THEN
30     SIGNAL SQLSTATE '45000'
31     SET MESSAGE_TEXT = 'Ricorrenza non coerente con la transazione';
32  END IF;
33 END$$

```

```
34 DELIMITER ;
```

Listing 6: Trigger di controllo sull'inserimento di una transazione

Le viste forniscono risultati già aggregati alla dashboard e alla vista amministrativa. In particolare, la vista amministrativa non espone le singole descrizioni né l'identità degli utenti, ma soltanto conteggi e somme per mese.

```
1 CREATE VIEW v_statistiche_aggregate_admin AS
2 SELECT
3     P.Anno ,
4     P.Mese ,
5     COUNT(*) AS Numero_Transazioni ,
6     SUM(CASE WHEN T.TipoTransazione = 'S'
7         THEN T.Importo ELSE 0 END) AS Totale_Spese ,
8     SUM(CASE WHEN T.TipoTransazione = 'E'
9         THEN T.Importo ELSE 0 END) AS Totale_Entrate
10 FROM TRANSIZIONE T
11 JOIN PERIODO P ON T.ID_Periodo = P.ID_Periodo
12 GROUP BY P.Anno , P.Mese ;
```

Listing 7: Vista delle statistiche aggregate per l'amministratore

```
1 CREATE VIEW v_budget_stato AS
2 SELECT
3     B.ID_Budget ,
4     B.Email ,
5     P.Mese ,
6     P.Anno ,
7     COALESCE(C.Nome, 'Budget globale') AS Ambito ,
8     B.Importo_Limite ,
9     COALESCE(SUM(T.Importo), 0) AS Totale_Speso ,
10    B.Importo_Limite - COALESCE(SUM(T.Importo), 0) AS Residuo ,
11    CASE WHEN COALESCE(SUM(T.Importo), 0) > B.Importo_Limite
12        THEN TRUE ELSE FALSE END AS Superato
13 FROM BUDGET B
14 JOIN PERIODO P ON B.ID_Periodo = P.ID_Periodo
15 LEFT JOIN CATEGORIA C ON B.ID_Categoria = C.ID_Categoria
16 LEFT JOIN TRANSIZIONE T
17     ON T.Email = B.Email
18     AND T.ID_Periodo = B.ID_Periodo
19     AND T.TipoTransazione = 'S'
20     AND (B.ID_Categoria IS NULL OR T.ID_Categoria = B.ID_Categoria)
21 GROUP BY B.ID_Budget , B.Email , P.Mese , P.Anno ,
22          C.Nome , B.ID_Categoria , B.Importo_Limite ;
```

Listing 8: Vista dello stato dei budget

6.6 Popolamento iniziale del database

Lo script `doc/sql/02_popolamento_demo.sql` inserisce un dataset dimostrativo più ampio rispetto alla prima versione. Il popolamento mantiene due account studente e un account amministratore tecnico, crea i periodi da gennaio a luglio 2026 e registra movimenti da gennaio a giugno. Per ogni mese sono presenti più entrate e spese: lo studente principale ha circa dieci movimenti mensili, mentre il secondo studente ne ha almeno otto. In questo modo i grafici annuali, le ricorrenze, i budget e l'isolamento tra utenti possono essere verificati con dati meno artificiali. Gli identificatori generati vengono recuperati mediante query o `LAST_INSERT_ID()`, evitando valori numerici rigidi nelle associazioni successive.

```

1 INSERT INTO UTENTE (Email, Password, Nome, Cognome, Ruolo)
2 VALUES
3 ('studente1@mail.com', SHA2('password1', 512), 'Mario', 'Rossi', 'UTENTE
4   '),
5 ('studente2@mail.com', SHA2('password2', 512), 'Luca', 'Bianchi', '
6   UTENTE'),
7 ('admin@sage.com', SHA2('admin123', 512), 'Admin', 'Sistema', 'ADMIN');
8
9 INSERT INTO PERIODO (Mese, Anno)
10 VALUES
11 (1, 2026), (2, 2026), (3, 2026), (4, 2026),
12 (5, 2026), (6, 2026), (7, 2026);
13
14 -- Le ricorrenze vengono create prima delle transazioni,
15 -- così le rate già generate possono essere collegate.
16 INSERT INTO SPESA_RICORRENTE
17 (Nome, Importo_Previsto, Frequenza_Giorni, Data_Inizio,
18  Data_Prossima_Scadenza, Scadenza, ID_Categoria, Email)
19 VALUES
20 ('Affitto', 450.00, 30, '2026-01-15', '2026-07-15',
21  '2026-12-31', @cat_casa, 'studente1@mail.com'),
22 ('Netflix', 9.99, 30, '2026-01-23', '2026-07-23',
23  '2026-12-31', @cat_svago, 'studente1@mail.com');
24
25 INSERT INTO TRANSIZIONE
26 (TipoTransazione, Importo, Data, Descrizione, Email,
27  ID_Categoria, ID_Periodo, ID_Fonte, ID_Ricorrenza)
28 VALUES
29 ('E', 250.00, '2026-01-03', 'Aiuto famiglia gennaio',
30  'studente1@mail.com', NULL, @periodo_gennaio, @fonte_aiuto_famiglia,
31  NULL),
32 ('S', 450.00, '2026-01-15', 'Affitto',
33  'studente1@mail.com', @cat_casa, @periodo_gennaio, NULL,
34  @ricorrenza_affitto_s1),
35 ('S', 9.99, '2026-01-23', 'Netflix',
36  'studente1@mail.com', @cat_svago, @periodo_gennaio, NULL,
37  @ricorrenza_netflix_s1),
38 ('E', 200.00, '2026-02-07', 'Lavoretto weekend febbraio',
39  'studente2@mail.com', NULL, @periodo_febbraio, @fonte_lavoretto, NULL),
40 ('S', 320.00, '2026-02-15', 'Stanza in affitto',
41  'studente2@mail.com', @cat_casa_fuori_sede_s2, @periodo_febbraio, NULL,
42  @ricorrenza_affitto_s2);

```

Listing 9: Estratto del popolamento esteso con due utenti e movimenti distribuiti nel tempo

6.7 Migrazione di un database esistente

Lo script `doc/sql/03_migrazione_database_esistente.sql` aggiorna in modo incrementale le installazioni create con versioni precedenti. Prima di ogni modifica consulta `INFORMATION_SCHEMA`; in questo modo aggiunge soltanto colonne, vincoli e indici ancora assenti. La migrazione rimuove anche eventuali classificazioni dimostrative residue create da vecchie esecuzioni dello script delle query operative, così tali voci non compaiono nell'interfaccia. Poiché la migrazione comprende aggiornamenti massivi controllati, salva il valore di `SQL_SAFE_UPDATES`, lo disattiva esclusivamente per la sessione corrente e lo ripristina al termine. In questo modo lo script è eseguibile anche da MySQL Workbench con la modalità sicura attiva, senza modificare le preferenze globali.

```

1 SET @sage_old_sql_safe_updates = @@SESSION.SQL_SAFE_UPDATES;
2 SET SESSION SQL_SAFE_UPDATES = 0;

```

```

3
4 -- aggiornamenti e cancellazioni controllati della migrazione
5
6 SET SESSION SQL_SAFE_UPDATES = @sage_old_sql_safe_updates;

```

Listing 10: Gestione della modalità safe update nella migrazione

```

1 SET @column_exists = (
2     SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS
3     WHERE TABLE_SCHEMA = DATABASE()
4           AND TABLE_NAME = 'CATEGORIA'
5           AND COLUMN_NAME = 'Icona'
6 );
7 SET @sql = IF(
8     @column_exists = 0,
9     'ALTER TABLE CATEGORIA ADD COLUMN Icona VARCHAR(255) NULL AFTER Nome',
10    'SELECT ''CATEGORIA.Icona gia presente'' AS Messaggio'
11 );
12 PREPARE stmt FROM @sql;
13 EXECUTE stmt;
14 DEALLOCATE PREPARE stmt;
15
16 SET @column_exists = (
17     SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS
18     WHERE TABLE_SCHEMA = DATABASE()
19           AND TABLE_NAME = 'TRANSIZIONE'
20           AND COLUMN_NAME = 'ID_Ricorrenza'
21 );
22 SET @sql = IF(
23     @column_exists = 0,
24     'ALTER TABLE TRANSIZIONE ADD COLUMN ID_Ricorrenza INT NULL AFTER ID_Fonte',
25     'SELECT ''TRANSIZIONE.ID_Ricorrenza gia presente'' AS Messaggio'
26 );
27 PREPARE stmt FROM @sql;
28 EXECUTE stmt;
29 DEALLOCATE PREPARE stmt;

```

Listing 11: Migrazione condizionale delle icone e del collegamento alle ricorrenze

6.8 Query SQL delle operazioni principali

6.8.1 Problemi e criticità

Le query devono garantire:

- isolamento utente tramite filtro sull'email;
- atomicità delle operazioni complesse;
- aggiornamento coerente delle ridondanze;
- protezione dalla divisione per zero nelle statistiche;
- validità dei documenti rispetto alle sole spese dell'utente.

6.8.2 Inserimento atomico di una spesa

```

1 START TRANSACTION;
2
3 INSERT INTO PERIODO (Mese, Anno)
4 VALUES (?, ?)

```

```

5 ON DUPLICATE KEY UPDATE ID_Periodo = LAST_INSERT_ID(ID_Periodo);
6
7 INSERT INTO TRANSIZIONE
8 (TipoTransazione, Importo, Data, Descrizione, Email, ID_Categoria,
9  ID_Periodo, ID_Fonte)
10 VALUES ('S', ?, ?, ?, ?, ?, LAST_INSERT_ID(), NULL);
11
12 SET @id_spesa = LAST_INSERT_ID();
13
14 INSERT INTO SPESA_TAG (ID_Transizione, ID_Tag)
15 VALUES (@id_spesa, ?);
16
17 UPDATE BUDGET
18 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
19 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria IS NULL;
20
21 UPDATE BUDGET
22 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
23 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria = ?;
24 COMMIT;

```

Listing 12: Inserimento di una spesa con periodo, tag e budget

6.8.3 Upsert del budget

```

1 SELECT COALESCE(SUM(Importo), 0) AS Totale_Speso
2 FROM TRANSIZIONE
3 WHERE Email = ?
4    AND ID_Periodo = ?
5    AND TipoTransazione = 'S'
6    AND (? IS NULL OR ID_Categoria = ?);
7
8 INSERT INTO BUDGET
9 (Importo_Limite, Alert_Soglia, Totale_Speso_Attuale, ID_Periodo,
10  ID_Categoria, Email)
11 VALUES (?, ?, ?, ?, ?, ?)
12 ON DUPLICATE KEY UPDATE
13 Importo_Limite = VALUES(Importo_Limite),
14 Alert_Soglia = VALUES(Alert_Soglia),
15 Totale_Speso_Attuale = VALUES(Totale_Speso_Attuale);

```

Listing 13: Upsert del budget con totale iniziale coerente

6.8.4 Documenti dell'utente

```

1 SELECT D.ID_Documento, D.ID_Transizione, D.Path_File, D.Tipo_File,
2        D.Data_Acquisizione_Documento,
3        T.Data, T.Importo, T.Descrizione
4 FROM DOCUMENTO D
5 JOIN TRANSIZIONE T ON D.ID_Transizione = T.ID_Transizione
6 WHERE T.Email = ?
7    AND T.TipoTransazione = 'S'
8 ORDER BY T.Data DESC, D.ID_Documento DESC;

```

Listing 14: Documenti associati alle spese dell'utente

```

1 SELECT T.ID_Transizione, T.Data, T.Importo, T.Descrizione
2 FROM TRANSIZIONE T
3 LEFT JOIN DOCUMENTO D ON D.ID_Transizione = T.ID_Transizione
4 WHERE T.Email = ?
5     AND T.TipoTransazione = 'S'
6     AND D.ID_Documento IS NULL
7 ORDER BY T.Data DESC, T.ID_Transizione DESC;

```

Listing 15: Spese dell'utente ancora prive di documento

6.8.5 Analisi percentuale

```

1 SELECT
2     (
3         SUM(CASE WHEN TipoTransazione = 'S' THEN Importo ELSE 0 END)
4         /
5         NULLIF(SUM(CASE WHEN TipoTransazione = 'E' THEN Importo ELSE 0
6         END), 0)
7     ) * 100 AS Percentuale_Spese_Su_Entrate
8 FROM TRANSIZIONE
9 WHERE Email = ? AND ID_Periodo = ?;

```

Listing 16: Rapporto spese/entrate mensile

6.8.6 Query di supporto

```

1 SELECT ID_Categoria, Nome, is_system
2 FROM CATEGORIA
3 WHERE is_system = TRUE OR Email_Proprietario = ?
4 ORDER BY is_system DESC, Nome;

```

Listing 17: Categorie disponibili per utente

```

1 SELECT T.ID_Transizione, T.Importo, T.Data, T.Descrizione,
2     COALESCE(C.Nome, F.Nome) AS Categoria_Fonte,
3     CASE WHEN T.TipoTransazione = 'S' THEN 'Spesa' ELSE 'Entrata' END
4     AS Tipo
5 FROM TRANSIZIONE T
6 LEFT JOIN CATEGORIA C ON T.ID_Categoria = C.ID_Categoria
7 LEFT JOIN FONTE F ON T.ID_Fonte = F.ID_Fonte
8 WHERE T.Email = ? AND T.Data BETWEEN ? AND ?
9 ORDER BY T.Data DESC;

```

Listing 18: Riepilogo transazioni per periodo

```

1 SELECT ID_Budget, Mese, Anno, Ambito,
2     Importo_Limite, Totale_Speso, Residuo, Superato
3 FROM v_budget_stato
4 WHERE Email = ?
5 ORDER BY Anno DESC, Mese DESC, Ambito;

```

Listing 19: Budget con residuo e stato di superamento

6.8.7 Motivazioni

L'uso di `PreparedStatement` evita concatenazioni insicure e permette di parametrizzare email, date e importi. L'uso di transazioni JDBC garantisce atomicità sulle operazioni che coinvolgono più tabelle. Le query sui documenti attraversano sempre `TRANSIZIONE`, così il filtro sull'utente rimane centralizzato.

7 Progettazione dell'applicazione

7.1 Architettura generale dell'applicazione

L'applicazione S.A.G.E. è un programma desktop organizzato secondo un'architettura a livelli. La separazione delle responsabilità evita che l'interfaccia grafica esegua direttamente le query e rende più semplice modificare una parte del sistema senza intervenire sulle altre. Il flusso ordinario è il seguente:

View → Controller → Service → DAO → MySQL

Le classi del livello **view** raccolgono gli input e aggiornano le schermate; i controller espongono le operazioni richieste dall'interfaccia; i service applicano le regole di dominio e coordinano le transazioni; i DAO contengono il codice JDBC e traducono le righe del database in oggetti del model. Le classi del package **model** rappresentano invece gli oggetti persistenti, come utente, transazione, budget, categoria, documento e spesa ricorrente.

La finestra principale **MainFrame** utilizza un **CardLayout**. All'avvio mostra il pannello di autenticazione; dopo il login crea una dashboard diversa in base al ruolo dell'utente. Un account con ruolo **UTENTE** accede alla dashboard personale, mentre un account con ruolo **ADMIN** accede alla dashboard amministrativa. Il database risiede in locale e viene raggiunto direttamente dall'applicazione, realizzando quindi un'architettura desktop 2-tier.

7.2 Tecnologie usate

Le tecnologie principali sono:

- **Java SE**: linguaggio usato per logica applicativa, accesso ai dati e interfaccia;
- **Java Swing e AWT**: costruzione delle finestre, dei form, delle tabelle, dei pannelli e dei grafici;
- **JDBC**: collegamento al DBMS e gestione di query parametrizzate e transazioni;
- **MySQL Connector/J**: driver JDBC incluso nella cartella **lib**;
- **MySQL**: DBMS utilizzato per la persistenza dei dati;
- **XAMPP**: ambiente locale usato per eseguire MySQL, gestire l'avvio del servizio e amministrare il database tramite phpMyAdmin;
- **SQL**: definizione dello schema, trigger, viste, popolamento e interrogazioni;
- **API Desktop di Java**: apertura dei documenti digitali con il programma predefinito del sistema operativo.

Non viene utilizzato un ORM: la traduzione tra oggetti Java e relazioni è gestita esplicitamente dai DAO. Questa scelta rende visibili le query effettivamente eseguite ed è coerente con gli obiettivi del corso.

7.3 Struttura dei package Java

Il codice sorgente si trova sotto **src/it/unibo/sage** ed è suddiviso come segue.

Package	Responsabilità
<code>it.unibo.sage</code>	Contiene il punto di ingresso Main .
<code>model</code>	Rappresenta le entità del dominio e i tipi enumerativi.
<code>dao</code>	Definisce le interfacce di persistenza e le implementazioni JDBC.
<code>service</code>	Implementa regole di dominio, aggregazioni e operazioni atomiche.
<code>controller</code>	Fa da collegamento tra pannelli Swing e servizi applicativi.

Package	Responsabilità
<code>view</code>	Contiene la finestra principale e le viste dedicate a login, dashboard, amministrazione, documenti e configurazione iniziale.
<code>view.components</code>	Componenti grafici riutilizzabili, come sidebar, pulsanti, pannelli di riepilogo e notifiche.
<code>view.dashboard</code>	Dashboard dell'utente, grafici e supporto alle icone di categorie, tag e fonti.
<code>view.admin</code>	Dashboard amministrativa e visualizzazione delle statistiche aggregate.
<code>utils</code>	Connessione al database e gestione della memorizzazione delle icone personalizzate.

Questa struttura segue la separazione in livelli adottata nelle relazioni di riferimento: le finestre non incorporano la logica SQL, ma delegano a classi dedicate che rendono esplicito il percorso tra interfaccia e database.

7.4 Collegamento al database tramite JDBC

La classe `DatabaseConnection` centralizza la creazione delle connessioni. Prima carica il driver `com.mysql.cj.jdbc.Driver`, quindi usa `DriverManager.getConnection`. Il server MySQL viene avviato localmente tramite XAMPP; in assenza di configurazioni esterne, il collegamento predefinito punta al database locale:

```
1 jdbc:mysql://localhost:3306/
2 Schema_finale_del_relazionale_SAGE_Vista_ibrida_Raffinata
3 ?serverTimezone=Europe/Rome
```

Listing 20: Configurazione predefinita della connessione JDBC

URL, utente e password possono essere sovrascritti tramite proprietà Java o variabili d'ambiente: `sage.db.url`, `sage.db.user`, `sage.db.password` oppure `SAGE_DB_URL`, `SAGE_DB_USER`, `SAGE_DB_PASSWORD`. Questo permette di eseguire lo stesso programma su configurazioni differenti senza modificare il codice sorgente.

Le connessioni, i `PreparedStatement` e i `ResultSet` sono chiusi con *try-with-resources*. Le query ricevono i dati attraverso parametri, evitando la concatenazione di stringhe. Le operazioni che coinvolgono più relazioni, come inserimento di una spesa, aggiornamento dei budget e generazione delle ricorrenze, disabilitano temporaneamente l'auto-commit e terminano con `commit`; in caso di errore viene eseguito il `rollback`.

7.5 Organizzazione delle classi principali

Classe o gruppo	Compito principale
<code>Main</code> , <code>MainFrame</code>	Avvio del programma, applicazione del tema, gestione del <code>CardLayout</code> e scelta della dashboard in base al ruolo.
<code>LoginPanel</code> , <code>LoginController</code> , <code>LoginService</code>	Autenticazione, registrazione di un nuovo studente e apertura della sessione.
<code>DashboardPanel</code>	Navigazione dell'utente tra riepilogo, transazioni, budget, categorie e tag, fonti, ricorrenze e documenti.
<code>AdminDashboardPanel</code> , <code>AdminService</code>	Indicatori globali, grafici mensili, categorie più utilizzate e consultazione delle viste aggregate.

Classe o gruppo	Compito principale
MovimentiController, MovimentiService	Inserimento, modifica, eliminazione e ricerca delle transazioni e delle classificazioni personali.
BudgetController, BudgetService	Creazione o aggiornamento dei budget e calcolo dello stato di utilizzo.
SpeseRicorrentiController, SpeseRicorrentiService	Gestione dei modelli ricorrenti e generazione delle transazioni mancanti fino alla data corrente.
DocumentiController, DocumentiPanel	Associazione, modifica, eliminazione e apertura dei documenti collegati alle spese.
DashboardDataService, DashboardOverviewCalculator	Raccolta dei dati necessari al riepilogo e calcolo di saldi e aggregazioni mostrate nei grafici.
Jdbc*DAO	Esecuzione delle query SQL e conversione dei risultati negli oggetti del model.

Le classi model non dipendono da Swing né da JDBC. Tale indipendenza permette di utilizzare gli stessi oggetti sia nelle viste sia nei servizi e semplifica i test della logica di calcolo.

7.6 Gestione degli errori e validazione input

La validazione è distribuita su più livelli:

- i form Swing verificano campi obbligatori, importi positivi, date e selezioni richieste;
- i service applicano regole di dominio, come unicità dei nomi disponibili, frequenza positiva delle ricorrenze e coerenza tra data iniziale, prossima scadenza e data finale;
- i DAO usano query parametrizzate e filtrano sempre per l'email dell'utente quando trattano dati personali;
- il DBMS applica chiavi esterne, CHECK, UNIQUE, trigger e vincoli di proprietà.

Gli errori di input vengono rappresentati principalmente con `IllegalArgumentException`; gli errori di accesso ai dati propagano una `SQLException`. Le viste intercettano tali eccezioni e mostrano messaggi mediante `JOptionPane`. Prima delle eliminazioni viene richiesta una conferma esplicita. I documenti sono controllati sia prima della registrazione sia prima dell'apertura, così da gestire il caso in cui il file sia stato spostato o cancellato.

Le operazioni più lente, come login, caricamento dei documenti e configurazione iniziale, usano `SwingWorker`: il lavoro JDBC viene eseguito fuori dall'Event Dispatch Thread e l'interfaccia resta utilizzabile. La password non viene salvata in chiaro, ma confrontata e registrata mediante SHA2 nel database.

7.7 Interfaccia grafica Java Swing

L'interfaccia è realizzata interamente in Swing. Un tema comune definisce font, spaziature e componenti; la sidebar permette di cambiare area senza aprire nuove finestre. La dashboard dell'utente utilizza pannelli a scheda per mostrare saldo, entrate, spese e percentuale di budget utilizzato. I grafici sono disegnati da componenti dedicati e si aggiornano quando l'utente seleziona il filtro mese, anno o intero storico.

Le principali aree disponibili all'utente sono:

- **Riepilogo:** indicatori, andamento temporale, distribuzione per categoria, stato dei budget e ultimi movimenti;
- **Transazioni:** schede delle entrate e delle spese, con modifica ed eliminazione;
- **Budget:** visualizzazione dei budget unici tramite barre di avanzamento, con modifica, eliminazione e creazione di nuovi limiti tramite il pulsante *+ Budget*;
- **Categorie e Tag e Fonti:** elementi di sistema e personali, con scelta o caricamento di un'icona e pulsanti di modifica/eliminazione;

- **Ricorrenze:** modelli periodici e consultazione delle spese generate;
- **Documenti:** documenti collegati alle spese, apribili con il programma predefinito.

La dashboard amministrativa è separata da quella personale. Mostra numero di utenti, numero di transazioni, spese, entrate, budget superati, andamento mensile globale e categorie più utilizzate. I dati sono aggregati: l'amministratore non accede alle descrizioni dei movimenti né ai documenti personali.

7.8 Schermate principali

La Figura 6 mostra la schermata principale dell'utente con il filtro annuale selezionato. Nella parte superiore sono riepilogati saldo, entrate, spese e budget utilizzato. Il grafico a barre confronta entrate e spese mese per mese, mentre il grafico ad anello mostra la distribuzione delle uscite tra le categorie. Nella parte inferiore sono riportati lo stato dei budget, rappresentato con barre percentuali, e le transazioni più recenti. La stessa rappresentazione a barre viene riutilizzata anche nella sezione dedicata alla gestione dei budget, che non mostra più una semplice tabella ma una vista più coerente con il riepilogo.



Figura 6: Dashboard principale dell'utente con riepilogo annuale.

La schermata costituisce il punto di accesso alle funzioni operative: il pulsante *Nuova transazione* apre il form di inserimento, i pulsanti *Mese*, *Anno* e *Tutto* cambiano l'intervallo analizzato e la sidebar permette di raggiungere le altre sezioni.

7.9 Flussi d'uso principali

1. **Registrazione e login:** lo studente crea un account, accede con email e password e viene indirizzato alla propria dashboard; il ruolo amministrativo apre invece la vista aggregata.
2. **Inserimento di una spesa:** l'utente specifica importo, data, descrizione e categoria, seleziona eventuali tag e documento; il sistema registra il periodo e aggiorna i budget interessati nella stessa transazione JDBC.

3. **Inserimento di una entrata:** l'utente seleziona una fonte; la transazione viene salvata senza categoria, tag o documento.
4. **Definizione di un budget:** l'utente apre il form tramite *+ Budget*, sceglie limite ed eventuale categoria; se esiste già un budget per lo stesso ambito, il sistema lo aggiorna invece di crearne una copia. Il totale speso viene ricalcolato sui movimenti del mese corrente e la schermata mostra la barra di avanzamento aggiornata.
5. **Gestione delle classificazioni:** categorie, tag e fonti personali possono essere creati, modificati o eliminati; nella versione dimostrativa sono esposti anche i pulsanti di modifica ed eliminazione sugli elementi di sistema, restando comunque attivi i vincoli referenziali del database.
6. **Spesa ricorrente:** il modello memorizza nome, importo, frequenza, categoria e scadenze; il servizio genera le transazioni non ancora registrate e ne evita la duplicazione.
7. **Documento digitale:** l'utente seleziona una spesa priva di allegato e un file locale; il DB conserva il percorso e l'interfaccia permette successivamente di aprirlo, modificarlo o eliminarlo.
8. **Statistiche amministrative:** l'amministratore consulta indicatori e grafici costruiti su dati aggregati e gestisce categorie, tag e fonti di sistema.

8 Test e validazione

8.1 Dati di test utilizzati

I dati di test includono due studenti normali e un amministratore. Lo studente `studente1@mail.com` ha transazioni, categorie personali, fonti, tag, budget, ricorrenze e almeno un documento dimostrativo. Lo studente `studente2@mail.com` consente di verificare l'isolamento dei dati con un secondo portafoglio completo. Il popolamento copre i mesi da gennaio a giugno 2026 con movimenti distribuiti nel tempo: sono presenti sia entrate sia spese, comprese rate ricorrenti collegate ai relativi modelli.

8.2 Test delle query SQL

Test	Input	Risultato atteso
Login valido	<code>studente1@mail.com</code> , <code>password1</code>	Utente trovato.
Login non valido	Password errata	Nessun utente trovato.
Documenti utente	Email studente 1	Solo documenti delle sue spese.
Spese senza documento	Email studente 1	Solo spese prive di documento.
Budget stato	Email studente 1	Budget unici per ambito, con stato calcolato sul periodo corrente.
Isolamento utenti	Email studente 2	Nessun dato personale dello studente 1.

8.3 Test delle funzionalità applicative

- avvio dell'applicazione da Main;
- login con credenziali demo;
- apertura della dashboard personale;
- visualizzazione della sezione documenti;
- apertura di un documento con path valido;
- gestione di path non valido con messaggio di errore;
- inserimento di un documento su una spesa senza documento.

8.4 Casi limite gestiti

- assenza di budget configurati;
- assenza di documenti per un utente;
- file documento rimosso dal disco;
- assenza di entrate nel calcolo percentuale;
- tentativo di associare un documento a una spesa non appartenente all'utente.

8.5 Errori noti o limitazioni

La principale limitazione riguarda i documenti: il DB conserva il percorso del file, non il contenuto. Se il file viene spostato, cancellato o se l'utente usa un altro dispositivo, il percorso può risultare non valido. Questa scelta è consapevole e motivata dalla semplicità del progetto e dal minor carico sul database.

9 Conclusioni

9.1 Risultato ottenuto

Il progetto realizza una base di dati relazionale coerente con il dominio della gestione economica personale. L'applicazione Java consente di autenticare un utente, mantenere una sessione, interrogare dati filtrati per email e visualizzare le principali aree funzionali.

9.2 Scelte progettuali principali

Le scelte più rilevanti sono il collasso della gerarchia spesa/entrata in TRANSIZIONE, l'isolamento dati tramite email, la modellazione ibrida di elementi globali/personali, la ridondanza controllata del budget e la memorizzazione dei documenti tramite path.

9.3 Possibili sviluppi futuri

Sviluppi futuri includono:

- sincronizzazione o copia controllata dei documenti in una cartella applicativa;
- eventuale uso di `LONGLOB` per rendere i documenti portabili attraverso il solo database;
- completamento delle funzionalità amministrative;
- grafici statistici più avanzati;
- esportazione dei report in PDF o CSV.